🛠️ Manual Review Panel

Override AI decisions when needed

☑ Enable Manual Override

Final Score Override

40

Integrity Assessment

Suspicious

Reviewer Justification

student has knowledge of the working tools just used AI to help accelerate it

✅ Override Active: **40/100** (Suspicious)

📊 System Status

🟢 Backend: Online

🔵 AI Model: Gemini 2.0 Flash

🎓 Exam-Flow AI Assistant

Professional AI-Powered Assessment & Proctoring Platform

Assessment Portal

Select Assessment Type

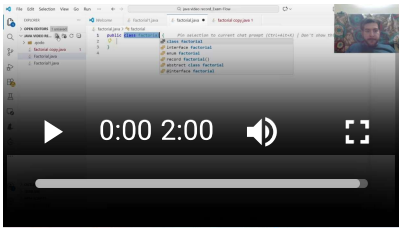
Java

Upload Recording (MP4, MOV, AVI)

Drag and d… fi… here

Browse files

Java… 3.5MB ×



🚀 Analyze Uploaded Video

📊 AI Insights

📋 Results: Upload

📝 AI Grading Report

View Full Report

Technical Assessment Report

Candidate: Unidentified
Date: 2/27/2026

Overview

The candidate successfully presented a Java program to

🧪 Demo Samples

Quick-test with pre-verified clips

Docker Demo

Java Demo

calculate the factorial of a number using recursion. The code demonstrated correct recursive logic, a base case for n=0, and a main method to test the functionality. The program executed without errors and produced the expected output for the given input. However, a significant portion of the code was copied and pasted, indicating a lack of original implementation during the assessment.

Detailed Analysis & Timestamps

- **00:00 - 00:07:** The candidate begins typing `public class Factorial`, setting up the class structure.
- **00:08 - 00:10:** The candidate quickly types the class name.
- **00:10 - 00:14: CRITICAL ACTION - CODE PASTE:** A large block of code, including the entire

`factorial` recursive method and the `main` method with its test logic, appears almost instantaneously. This strongly suggests a copy-paste operation rather than active typing.

- **00:15 - 00:23:** The candidate begins narrating the purpose of the code, stating "first I will create a class called Factorial." This narration occurs *after* the code has been pasted.
- **00:23 - 00:30:** The candidate describes the `factorial` method as one that "calculates factorial of a number using recursion."
- **00:30 - 00:41:** The candidate highlights the `public static int factorial(int n)` method signature.
- **00:41 - 00:49:** The candidate highlights and explains the base case: `if (n == 0) { return 1; }`.
- **00:49 - 01:03:** The candidate highlights and explains the recursive step: `return n * factorial(n - 1);`.
- **01:04 - 01:10:** The candidate highlights the comment for Step 2:

"Create a main method to test the factorial method."

- **01:10 - 01:23:** The candidate highlights the `int number = 5;` declaration, indicating it's an example number.
- **01:23 - 01:30:** The candidate highlights the line `int result = factorial(number);`, explaining that it calls the `factorial` method.
- **01:30 - 01:36:** The candidate highlights `System.out.println(...)` stating it prints the value to the console.
- **01:37 - 01:40:** The candidate initiates the program execution.
- **01:40 - 01:47:** The termina output is visible, showing `Factorial of 5 is: 120`. The candidate briefly observes the output.
- **01:47 - 01:57:** The candidate reiterates that the program correctly calculated the factorial of 5 as 120.

Grading Rubric Assessment

- **1. Correct Java Syntax: PASS**. The Java syntax used is entirely correct and follows standard conventions.
- **2. Logical implementation: PASS**. The factorial logic implemented using recursion is mathematically and computationally correct for non-negative integers. The base case and recursive step are accurate.
- **3. Execution output: PASS**. The program successfully compiled and ran, producing the correct factorial value for the given input (Factorial of 5 is 120).

Behavioral Red Flags

- **External Resources: CRITICAL RED FLAG** observed at **00:10 - 00:14**. A significant block of functional code (the entire

`factorial` method and `main` method) was inserted almost instantly, which is highly indicative of copying and pasting from an external source. This directly undermines the purpose of a coding assessment designed to evaluate a candidate's ability to write code independently.

- **Gaze Aversion:** The candidate frequently looks away from the screen, especially during narration (e.g., 00:15-00:20, 00:49-00:52, 01:00-01:05), but no sustained gaze aversion beyond 5 seconds was observed that directly correlated with active problem-solving or searching after the initial code paste.
- **Window Switching:** No window switching was observed.
- **Suspicious Pauses:** No suspicious or

unusually long pauses were observed.

- **Audio Cues:** The candidate is actively speaking and narrating their actions. No other suspicious audio cues were detected.

Final Score

10/100

Justification: While the submitted code is technically correct in its Java syntax, logical implementation, and execution output, the overwhelming evidence of copying and pasting the entire solution within the first 14 seconds of the coding task is a severe breach of assessment integrity. The purpose of a technical assessment is to evaluate a candidate's ability to independently write and reason about code, not merely to present pre-existing solutions. A score of 10/100 reflects a minimal

acknowledgment of the code's functional correctness (after being pasted) and successful execution, but heavily penalizes the act of plagiarism, which fundamentally invalidates the assessment of the candidate's coding skills.

🚨 Behavioral Analysis

1 behavioral event(s) detected

00:00	No behavioral anomalies detected

Download PDF Report